HeatGate: 3-Day Build Plan

Oct 6, 2026 · @Satya

What we are building

HeatGate turns heat-safety rules for delivery riders into code that runs inside the order dispatch system. Every time an order is about to be assigned, HeatGate checks the rider's heat exposure against a rule set and returns allow, skip this order, pause, or take a cooling break (15 or 20 minutes, depending on the rule set) with no rating penalty. Every decision is written to an audit log that unions and regulators can search.

The demo in one sentence: 50 simulated riders in a city, a heatwave slider, and four rule sets (today's voluntary measures, NDMA advisory rules, union demands, and our own HeatGate Fair), showing who gets rested, why, and what each costs in delivery time and in rider earnings.

Three screens, one app:

1. **Policy sandbox** (the main screen): heat slider, policy toggle, city map of riders, and side-by-side metrics for all four rule sets.
2. **Rider phone view**: what one rider sees. Either their next order, or "Rest 20 minutes, nearest shade and water here, no penalty."
3. **Audit log**: every decision with the rule that caused it. Filter by rider, decision or policy version.

In scope for 3 days

- Cedar policies for four rule sets, evaluated for real on every dispatch
- A simulator: 50 riders on staggered shifts, orders arriving, a simulated clock (1 real second = 1 simulated minute)
- Heat index computed from temperature and humidity, controlled by the slider
- Audit log with "which rule fired" for every decision
- Metrics comparing all four rule sets on the same simulated day, including rider earnings
- Deployed on AWS (Lambda, API Gateway, DynamoDB), with a public demo URL

Out of scope (say so in the pitch as "next steps")

- Real rider GPS or real delivery platform integration
- Live weather feed (instead, we replay a real past heatwave day, saved in the repo)

- Login and user accounts
- Mobile app (the rider view is a phone-sized web panel)

Important caution: the heat thresholds in this plan are placeholders for the demo. Before you present numbers as "safe" or "unsafe", have them checked against IMD and NDMA heat guidance, and say in the demo that thresholds are configurable.

Reality check: the real rules vs our model

The NDMA and union rule sets copy what the sources say, so a judge who knows them will not catch us out. HeatGate Fair is our own proposal. The numbers below come from news coverage of the NDMA gig-worker advisory (July 2025) and of the union demand. **Read the NDMA advisory itself before you pitch, and fix any number that differs.**

Source	What it says	How the plan models it
NDMA gig-worker advisory	Halt mandatory work from 11 AM to 4 PM during IMD Orange and Red alerts	`ndma-peak-hours-pause`, triggered by `alertLevel` 1 or more
NDMA	A 15-minute cooling break per 90 minutes of logged-in work, with no penalty or pay cut	`ndma-break-due` (rest 15). We apply it on alert days only; check the advisory for whether it applies every day
NDMA	A reduced task radius of 3 km	`ndma-reduced-radius` skips orders over 3 km on alert days
NDMA	It is a recommendation, and unions say it cannot be enforced	Our pitch: HeatGate is the missing enforcement and audit layer
Union demand (IFAT)	Paid cooling breaks of at least 20 minutes per 2 hours of recorded work on IMD heatwave days, plus protection from penalties, ID blocks and lost incentives	`union-rest-due` (rest 20, limit 120, alert days), penalty-free decline, and paid rest plus incentive protection in the simulator
Platforms	Some already add rest points, coolers or cooling vests (not individually verified)	The baseline is "today: voluntary measures only". We model only what is missing: enforcement and proof

Alert level. NDMA and the union trigger on IMD heat alerts, so the engine takes an `alertLevel` input: 0 none, 1 orange, 2 red. On a slider day it is 1 when the peak is 40 °C or more and 2 at 45 °C or more. This is a stand-in, because IMD's real criteria also use departure from normal. The screen shows "alert level (assumed)" and lets the judge override it. On a replayed real day, enter the real IMD alert by hand if you can find it.

Limits to state in the pitch

- **Heat index is a proxy.** It assumes shade and light wind. For outdoor work, WBGT (ISO 7243) is the better measure, because it includes sun and surface heat. Our thresholds are policy numbers (90/15 from NDMA, 120/20 from the union), not physiology.
- **One temperature for the whole city.** Heat while waiting outside, on other apps, or commuting is not counted.

- **Riders on several apps.** We model it (`multiAppPct`), but one platform cannot see another app. A shared limit needs a regulator-level system.
- **The heat fee is a hypothesis.** Customers may order less, and it may stack with state welfare fees on gig platforms. We have not checked either.
- **A log is only as good as the call.** A platform could skip HeatGate. Regulators would need independent evidence, such as audits.
- **Not a medical device.** Every threshold needs expert review.

Verify before you pitch

- [ ] Read the NDMA gig-worker advisory (July 2025) and quote its numbers from the text
- [ ] Confirm IFAT's 20-per-2-hours demand from a primary source
- [ ] Confirm each platform's current heat measures from primary sources before saying anything about them
- [ ] Check Karnataka, Rajasthan and Code on Social Security levies before calling the heat fee viable
- [ ] Ask a heat-physiology expert about riders' workload and WBGT
- [ ] Get legal advice on DPDP: whether pseudonymous IDs are personal data, and who may see the log
- [ ] Interview riders (questions in Built for Indian riders)

Built for Indian riders

A heat break that costs a rider money will not be taken. In India, riders will ignore it or log into another app. So HeatGate's own rule set, **HeatGate Fair**, protects riders without making safety a financial loss. The NDMA and union rule sets follow the real advisory and demand, and stay in the demo as comparisons, alongside a baseline of today's voluntary measures with nothing enforced.

Pitch line: "Safety that doesn't cost the rider a rupee of their day."

Ground reality	What goes wrong	How HeatGate Fair handles it	In the demo?
Rest means lost money	Riders skip breaks or switch apps	Rest is paid, and moved into quiet time where the rules allow	Yes

Ground reality	What goes wrong	How HeatGate Fair handles it	In the demo?
Daily incentive targets	Missing a ₹ bonus feels worse than missing one order	Forced rest or pause time reduces the day's target, so rest never costs the bonus	Yes
Being blocked feels like punishment	Riders resent the system	Soft steps first: skip long-distance orders, early rest in quiet time. Full stop only at extreme heat. Declining an order in heat never hurts the rating	Yes
Nowhere to rest	"Go rest" with no shade or water nearby	Rest happens at cooling points. Waiting at a restaurant counts as rest only if rider interviews confirm it has shade (switch is off by default)	Yes
Platforms need speed	Platforms reject anything that slows deliveries	Rest is delayed in a rush (up to a hard limit of 120 minutes) and long orders are skipped instead of stopping riders	Yes
Who pays for rest	Platforms refuse the cost	A small heat fee per order on hot days, shown next to the total rest pay	Yes (amounts are placeholders)

Ground reality	What goes wrong	How HeatGate Fair handles it	In the demo?
Riders on 2–3 apps	Paused on one app, riding on another	Modelled: about 40% of riders keep working on other apps while resting here (to be confirmed with riders), so their heat clock keeps running. A shared limit across platforms needs a regulator-level system (next step)	Modelled; cross-platform limit is pitch only
Language and literacy	English-only messages get ignored	Hindi, Telugu and English messages (Urdu if a native speaker helps), large text and icons. Later: voice notes	Hindi and Telugu yes, voice later
Cheap phones, patchy network	The rider screen doesn't load	Rider screen is tiny and text-first. Later: SMS fallback	Partly
Rider privacy (DPDP Act, 2023)	Audit logs could expose riders	Audit log stores a pseudonymous rider ID, never a name or phone number. This is still likely personal data under DPDP, since the platform can map the ID back, so access needs a legal basis and tiers	Yes

One honest trade-off to state in the pitch: Fair lets a break slip by up to 20 minutes during a rush (soft limit 100, hard limit 120 working minutes). The hard limit equals the union's demand of 120 and is 30 minutes looser than NDMA's 90. Whether that is acceptable is for health experts to decide. The sandbox exists to show exactly that trade-off.

Before Day 1: talk to 3–5 real riders (15 minutes total)

- [ ] How much do you earn per order, roughly?

- [ ] What daily or weekly incentive targets do you chase?
- [ ] Where do you wait between orders?
- [ ] What would make you actually take a break in the afternoon?

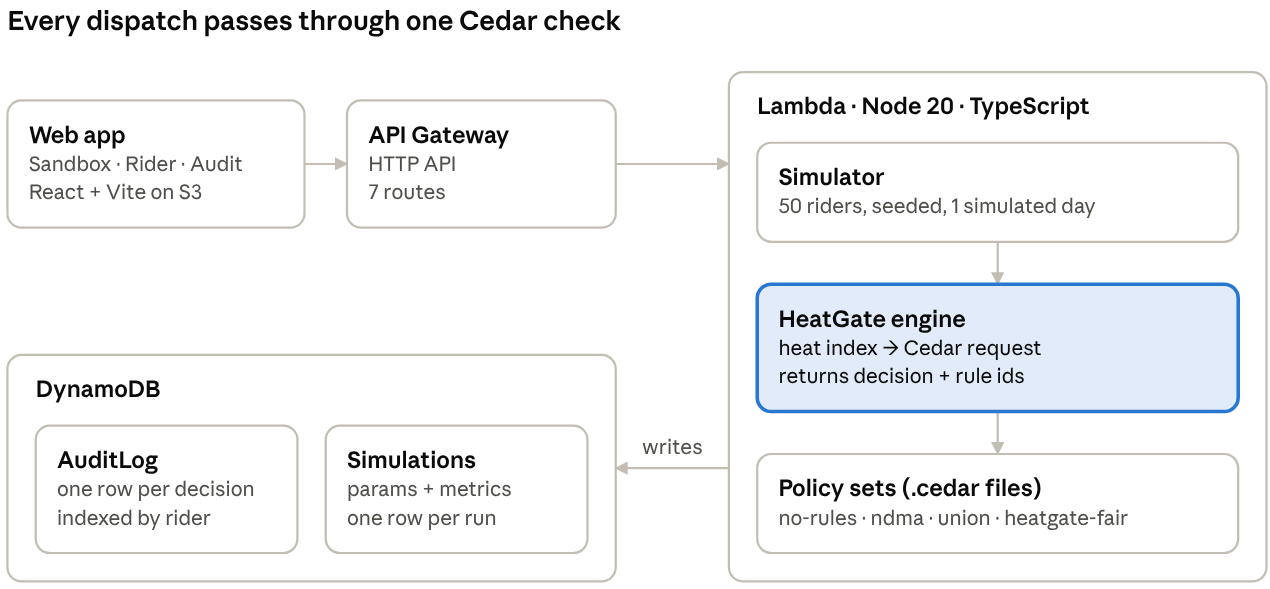
Also ask (these decide how the simulator is set up):

- [ ] Do you choose which orders to accept, or are they assigned to you?
- [ ] Do you work on more than one app at the same time? How many hours a day?
- [ ] Where do you really rest in the afternoon? Do the restaurants you visit have shade and water for riders?
- [ ] What do you fear more: a lower rating, being blocked, or losing an incentive slab?

Use their answers to replace the placeholder money amounts in the simulator, and quote them (no names, with permission) in the pitch. Real rider voices are worth more to judges than any feature.

Architecture

One Lambda function holds everything: the HeatGate engine (the product) and the simulator (the demo harness). The simulator calls the exact same engine function that `POST /decide` calls, so the demo proves the real thing works.



HeatGate architecture · 1 Lambda, 2 tables

The key design choice: the simulator runs a whole simulated day in one request (a few seconds), saves it as "frames", and the web app replays those frames as an animation. No long-running servers, no websockets, nothing that can break live on stage. Runs are seeded, so the same inputs always give the same result.

Tech stack

Layer	Choice	Why
Frontend	React + Vite + TypeScript + Tailwind	Fast to build, Claude writes it well
City map	Plain SVG grid (a 10 × 10 km box)	No map library to fight with; Leaflet is a stretch goal
Charts	Recharts	Simple bar and line charts for the metrics
Backend	Node.js 20 + TypeScript on AWS Lambda	One language for front and back
Policy engine	Cedar via the `@cedar-policy/cedar-wasm` npm package	Runs inside the Lambda, no extra service
API	API Gateway HTTP API	Cheapest, simplest
Storage	DynamoDB, on-demand	No server, free tier is enough
Infra as code	AWS SAM (one `template.yaml`)	One command deploys everything
Hosting	S3 static website (or Amplify Hosting)	Public demo URL
Local dev	Small Express server wrapping the same handlers, in-memory store	Build all of Day 1 without touching AWS

Repo layout (npm workspaces, one Git repo)

```
heatgate/
  CLAUDE.md                  # instructions Claude reads first (Section:
Building with Claude)
  contracts/
    types.ts                 # all shared TypeScript types (the contracts
below)
  policies/
    heatgate.cedarschema     # Cedar schema
```

```
    ndma-2025.cedar          # rule set A
    union-demand.cedar       # rule set B
    heatgate-fair.cedar      # rule set C, our own
    no-rules.cedar           # baseline: today's voluntary measures only
  engine/
    heatIndex.ts             # temp + humidity -> heat index
    decide.ts                # builds Cedar requests, returns Decision
    decide.test.ts           # golden test cases
  sim/
    city.ts  riders.ts  orders.ts  dispatcher.ts  metrics.ts  run.ts
  api/
    handlers/ decide.ts  simulations.ts  audit.ts  policies.ts
    store/    dynamo.ts  memory.ts
    local.ts                 # Express server for local dev
  web/                       # React app
  template.yaml              # AWS SAM
  README.md  LICENSE         # MIT, so the repo is open source
```

Contracts

These are the fixed agreements between the parts. Write them first (Day 1, hour 1), commit them, and do not change them without telling the whole team. Every Claude prompt later points back to these files.

1. Cedar schema (`policies/heatgate.cedarschema`)

Cedar only works with whole numbers (`Long`), so the engine rounds the heat index to the nearest °C, and the order distance up to a whole km, before sending them.

```
namespace HeatGate {
  entity Rider = {
    vehicle: String,          // "bike" | "cycle" | "ev"
  };
  entity Order = {
    zone: String,
    distanceKm: Long,         // drop distance, rounded UP to a whole km
  };
  type HeatContext = {
    heatIndexC: Long,             // rounded heat index at rider location, °C
    hourOfDay: Long,              // 0-23, local time
    minutesSinceBreak: Long,      // minutes worked since the last completed
```

```
rest
    exposureMinutesToday: Long,   // total outdoor minutes today
    restMinutesRemaining: Long,   // > 0 while a forced rest is running
    tripMinutes: Long,            // estimated outdoor minutes for this order
    demandLevel: Long,            // 0 = quiet, 1 = normal, 2 = rush (share
of riders busy)
    alertLevel: Long,             // 0 = none, 1 = IMD orange, 2 = IMD red
(assumed in the demo)
  };
  action AssignOrder appliesTo {
    principal: Rider, resource: Order, context: HeatContext
  };
  action DeclineWithoutPenalty appliesTo {
    principal: Rider, resource: Order, context: HeatContext
  };
}
```

2. Cedar policies: the four rule sets

Each `forbid` rule carries these annotations: `@id` (shown in the audit log), `@outcome` (SKIP_ORDER, DENY or FORCE_REST) and `@message` (shown to the rider). FORCE_REST rules also carry `@restMinutes` (the break length; 20 if missing). Translations are optional: `@messageHi`, `@messageTe`, `@messageUr`. The engine reads all of these at load time.

Rule set A: `policies/ndma-2025.cedar` (the NDMA July 2025 gig-worker advisory: pause 11 AM to 4 PM on orange and red alerts, a 15-minute break per 90 minutes of work, a 3 km radius). It is only a recommendation today. HeatGate is the layer that would enforce it.

```
@id("base-allow-assign")
permit (principal is HeatGate::Rider,
        action == HeatGate::Action::"AssignOrder",
        resource is HeatGate::Order);

@id("ndma-rest-in-progress")
@outcome("FORCE_REST")
@message("Your cooling break is still running. No penalty and no pay cut.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.restMinutesRemaining > 0 };

@id("ndma-peak-hours-pause")
@outcome("DENY")
@message("Heat alert: no mandatory orders from 11 AM to 4 PM. No penalty.")
```

```
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.alertLevel >= 1
       && context.hourOfDay >= 11 && context.hourOfDay < 16 };

@id("ndma-break-due")
@outcome("FORCE_REST")
@restMinutes("15")
@message("Heat alert: 15-minute cooling break after 90 minutes of work. No
penalty and no pay cut.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.alertLevel >= 1
       && context.minutesSinceBreak + context.tripMinutes > 90 };

@id("ndma-reduced-radius")
@outcome("SKIP_ORDER")
@message("Heat alert: orders are limited to 3 km.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.alertLevel >= 1 && resource.distanceKm > 3 };

@id("ndma-decline-no-penalty")
permit (principal, action == HeatGate::Action::"DeclineWithoutPenalty",
resource)
when { context.alertLevel >= 1 };
```

Rule set B: **`policies/union-demand.cedar`** (IFAT's demand: a paid cooling break of at least 20 minutes per 2 hours of recorded work on IMD heatwave days, plus protection from penalties and lost incentives)

```
@id("base-allow-assign")
permit (principal is HeatGate::Rider,
        action == HeatGate::Action::"AssignOrder",
        resource is HeatGate::Order);

@id("union-rest-in-progress")
@outcome("FORCE_REST")
@message("Your cooling break is still running. No penalty.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.restMinutesRemaining > 0 };

@id("union-rest-due")
@outcome("FORCE_REST")
@restMinutes("20")
@message("Heatwave day: paid 20-minute cooling break after 2 hours of work.
```

```
Nearest shade and water below. No penalty.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.alertLevel >= 1
       && context.minutesSinceBreak + context.tripMinutes > 120 };

@id("union-decline-no-penalty")
permit (principal, action == HeatGate::Action::"DeclineWithoutPenalty",
resource)
when { context.alertLevel >= 1 };
```

Why 120 and 20: 2 hours of work, then a 20-minute break. If taking this order would push the rider past 120 minutes since their last break, they rest first. NDMA's numbers are 90 and 15. The union demand has no extreme-heat stop, so this rule set has none either; that is what the demand says. The heat-index values used by Fair (38, 41, 46 °C) are demo placeholders, to be checked by a health expert.

Rule set C: **`policies/heatgate-fair.cedar`** (our own: protects riders without costing them money)

It adds one new outcome, `SKIP_ORDER`: this order is too far for the heat (more than 3 km), but the rider stays online for nearer orders. Paid rest and incentive protection are handled by the simulator (see Core logic), because they are about pay, not about whether an order is allowed. Unlike NDMA and the union, Fair reacts to the heat index directly instead of waiting for a daily alert.

```
@id("base-allow-assign")
permit (principal is HeatGate::Rider,
        action == HeatGate::Action::"AssignOrder",
        resource is HeatGate::Order);

@id("fair-rest-in-progress")
@outcome("FORCE_REST")
@message("Your paid cooling break is still running.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.restMinutesRemaining > 0 };

@id("fair-rest-early-in-lull")
@outcome("FORCE_REST")
@restMinutes("20")
@message("Orders are slow right now. Take a paid 20-minute break in the
shade.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
```

```
when { context.heatIndexC >= 38
       && context.minutesSinceBreak >= 70
       && context.demandLevel == 0 };

@id("fair-rest-due")
@outcome("FORCE_REST")
@restMinutes("20")
@message("Paid 20-minute cooling break. Nearest shade and water below.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.heatIndexC >= 38
       && context.minutesSinceBreak + context.tripMinutes > 100
       && context.demandLevel < 2 };

@id("fair-rest-hard-limit")
@outcome("FORCE_REST")
@restMinutes("20")
@message("Paid 20-minute cooling break. Nearest shade and water below.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.heatIndexC >= 38
       && context.minutesSinceBreak + context.tripMinutes > 120 };

@id("fair-short-trips-only")
@outcome("SKIP_ORDER")
@message("This order is too far for this heat. You stay online for nearer
orders.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.heatIndexC >= 41 && resource.distanceKm > 3 };

@id("fair-extreme-heat-stop")
@outcome("DENY")
@message("Extreme heat. Deliveries paused. Your incentive target is reduced
for this time.")
forbid (principal, action == HeatGate::Action::"AssignOrder", resource)
when { context.heatIndexC >= 46 };

@id("fair-decline-no-penalty")
permit (principal, action == HeatGate::Action::"DeclineWithoutPenalty",
resource)
when { context.heatIndexC >= 38 };
```

In plain words: rest early when it's quiet, rest on time when it's normal, delay rest in a rush but never past 120 minutes (the same as the union's limit, 30 looser than NDMA's), and in serious heat skip long-distance orders instead of stopping the rider.

Baseline: `policies/no-rules.cedar` (today: voluntary platform measures such as rest points and coolers, with nothing enforced at dispatch). The id stays `no-rules`; the screen shows the label from `POLICY_LABELS`.

```
@id("base-allow-assign")
permit (principal is HeatGate::Rider,
        action == HeatGate::Action::"AssignOrder",
        resource is HeatGate::Order);
```

No permit for `DeclineWithoutPenalty`, so declining counts against the rider. That is an assumption: ask riders how a declined order is treated today.

Language: every forbid rule can also carry `@messageHi` (Hindi), `@messageTe` (Telugu) and `@messageUr` (Urdu). Native speakers write these; do not machine-translate them for the demo. The demo needs Hindi, Telugu and English (Urdu if a native speaker is available). The engine falls back to `@message` when a translation is missing.

3. How the engine turns Cedar results into a decision

1. Ask Cedar: is `AssignOrder` allowed?
2. If allowed → outcome `ALLOW`.
3. If denied → take the forbid rules that fired. The strongest outcome wins: `FORCE_REST` beats `DENY`, which beats `SKIP_ORDER`. If several rules share the strongest outcome, the first one in policy-file order is the winner: it supplies the message and the rest length. The engine keeps each file's `@id` list in file order for this. `firedRules` lists the winner first, then the others.
4. Ask Cedar a second question: is `DeclineWithoutPenalty` allowed? That sets `penaltyWaived`. For `SKIP_ORDER`, `DENY` and `FORCE_REST`, `penaltyWaived` is always true.
5. Rest length: `restMinutesRemaining` if the winner's id ends in `-rest-in-progress`, otherwise the winner's `@restMinutes` (20 if missing).
6. Message: the winner's `@message`, plus any `@messageHi`, `@messageTe`, `@messageUr` it has.
7. The simulator, not Cedar, resets `minutesSinceBreak` to 0 when a rest finishes.

4. Shared types (`contracts/types.ts`)

```
export type PolicyVersion = "no-rules" | "ndma-2025" | "union-demand" |
"heatgate-fair";

// Names shown on screen. The id "no-rules" stays; the label says what it
really is.
export const POLICY_LABELS: Record<PolicyVersion, string> = {
  "no-rules": "Today: voluntary measures only (nothing enforced)",
  "ndma-2025": "NDMA advisory rules",
  "union-demand": "Union demand (IFAT)",
  "heatgate-fair": "HeatGate Fair",
};

// ALLOW      = give the order
// SKIP_ORDER = this order is too far for the heat; rider stays online for
nearer ones
// DENY       = rider paused, no orders for 15 minutes
// FORCE_REST = rider takes a cooling break (15 or 20 minutes, set by the
rule)
export type Outcome = "ALLOW" | "SKIP_ORDER" | "DENY" | "FORCE_REST";

export type AlertLevel = 0 | 1 | 2; // 0 none, 1 IMD orange, 2 IMD red
(assumed in the demo)

export interface DecideRequest {
  policyVersion: PolicyVersion;
  rider: { id: string; vehicle: "bike" | "cycle" | "ev"; x: number; y: number
}; // km in the 10x10 city
  order: { id: string; zone: string; distanceKm: number; tripMinutes: number
};
  conditions: { tempC: number; humidityPct: number; hourOfDay: number;
demandLevel: 0 | 1 | 2; alertLevel: AlertLevel };
  riderState: { minutesSinceBreak: number; exposureMinutesToday: number;
restMinutesRemaining: number };
  simulationId?: string;   // set when the simulator is calling
  dryRun?: boolean;        // true = do not write to AuditLog
}

export interface Decision {
  decisionId: string;      // ULID
  at: string;              // ISO time (simulated time inside a simulation)
  policyVersion: PolicyVersion;
```

```
  riderId: string;         // pseudonymous ID. Likely still personal data
(DPDP). Never a name or phone number
  orderId: string;
  simulationId?: string;
  outcome: Outcome;
  restMinutes: number;     // 0 unless FORCE_REST (15 or 20, or what is left
of a running rest)
  penaltyWaived: boolean;
  heatIndexC: number;      // rounded value sent to Cedar
  firedRules: string[];    // Cedar @id values: the winner first, then the
others that fired
  message: string;         // what the rider sees (English)
  messages?: { hi?: string; te?: string; ur?: string }; // translations, when
the rule has them
  coolingPoint?: { name: string; distanceM: number }; // nearest shade +
water, for FORCE_REST
  cedarContext: Record<string, number>; // exact inputs, so any decision can
be replayed
}

// All money amounts are PLACEHOLDERS for the demo. Replace with numbers from
rider interviews.
export interface Economics {
  payPerOrderInr: number;       // default 25
  payPerKmInr: number;          // default 6
  dailyTargetShare: number;     // incentive target = this share of the
median orders per rider in the no-rules run (same seed), default 0.9
  dailyBonusInr: number;        // paid if the target is reached, default 150
  restPayPerMinuteInr: number;  // paid where PayProtections.paidRest is on,
default 1.5 (= ₹30 per 20-minute break)
  heatFeePerOrderInr: number;   // heatgate-fair only, charged to customers
when heat index >= 38, default 5
}

// Which pay protections each rule set has. These defaults are ASSUMPTIONS to
verify:
// NDMA advises no penalty or pay cut; IFAT asks for paid breaks and
protection from lost incentives.
export interface PayProtections {
  paidRest: boolean;          // rest minutes are paid
  protectIncentive: boolean;  // every 30 minutes of forced rest or pause
lowers the daily target by 1 order
}
```

```
export const DEFAULT_PAY: Record<PolicyVersion, PayProtections> = {
  "no-rules":      { paidRest: false, protectIncentive: false },
  "ndma-2025":     { paidRest: true,  protectIncentive: false },
  "union-demand":  { paidRest: true,  protectIncentive: true  },
  "heatgate-fair": { paidRest: true,  protectIncentive: true  },
};

// Only for heatgate-fair
export interface FairFeatures {
  heatFee: boolean;                // charge heatFeePerOrderInr to customers
on hot orders. default true
  shadeWaitCountsAsRest: boolean;  // default FALSE. Assumes restaurants have
shade for riders, which is unverified
}

export type AlertSetting = "auto" | AlertLevel; // auto: 1 if peak >= 40 °C,
2 if peak >= 45 °C

export interface SimParams {
  seeds: number[];         // default [1,2,3,4,5]. Same seeds for every rule
set
  riders: number;          // default 50
  peakTempC: number;       // the slider, e.g. 34-46
  humidityPct: number;     // default 45
  alertLevel: AlertSetting;
  startHour: number;       // default 8
  endHour: number;         // default 20
  ordersPerHour: number;   // default 50; x1.5 at lunch (12-14) and dinner
(19-21). Tune so demand is quiet ~20%, normal ~50%, rush ~30% of the day
  shiftHours: { min: number; max: number }; // default 6 and 8; riders log in
at staggered times
  multiAppPct: number;     // default 40 (to confirm with riders): share of
riders who keep working on another app while resting here
  feeDemandDropPctPerInr: number; // default 1: % of new orders lost per
rupee of heat fee
  policyVersions: PolicyVersion[]; // usually all four
  economics: Economics;
  pay: Record<PolicyVersion, PayProtections>; // default DEFAULT_PAY
  fairFeatures: FairFeatures;
}

export type RiderState = "idle" | "delivering" | "resting" | "paused";
```

```
export interface Frame {
  minute: number;          // minutes since startHour
  heatIndexC: number;
  demandLevel: 0 | 1 | 2;
  riders: { id: string; x: number; y: number; state: RiderState }[];
  events: { riderId: string; outcome: Outcome; rule: string }[]; // decisions
in this step
}

export interface Metrics {
  // safety: the same yardstick for every rule set, whatever its own limit is
  unsafeExposureMin90: number;   // outdoor minutes at heat index >= 38
beyond 90 minutes since the last rest (NDMA's limit)
  unsafeExposureMin120: number;  // the same, beyond 120 minutes (the union's
limit)
  restsForced: number;
  pausedRiderMinutes: number;
  // speed
  ordersTotal: number;
  ordersDelivered: number;
  avgDeliveryMinutes: number;      // order created -> delivered
  p90DeliveryMinutes: number;
  ordersLate: number;              // delivered after 45 minutes
  // money
  avgEarningsPerRiderInr: number;  // order pay + bonus + rest pay
  minEarningsPerRiderInr: number;  // the worst-off rider
  incentiveHitRatePct: number;     // % of riders who reached their
(adjusted) target
  restPayTotalInr: number;
  heatFeeCollectedInr: number;
  decisions: Record<Outcome, number>;
}

export type NumericMetric = Exclude<keyof Metrics, "decisions">;

export interface SimResult {
  simulationId: string;
  params: SimParams;
  runs: Partial<Record<PolicyVersion, {
    metrics: Metrics;                                          // mean
over the seeds
    range: Record<NumericMetric, { min: number; max: number }>;    // across
the seeds
```

```
    frames: Frame[];                                                    // first
seed only, one per 5 sim minutes
  }>>;
}
```

`unsafeExposureMin90` and `unsafeExposureMin120` are measured the same way for every rule set, whatever limit it uses itself, so they are a fair yardstick: outdoor minutes at heat index 38 or more beyond 90 minutes (NDMA's limit) or 120 minutes (the union's limit) since the last rest. They are the "harm" numbers, and `avgDeliveryMinutes` is the "cost" number. Neither is a medical measure.

5. REST API

Method + path	Body / query	Returns	Notes
`POST /decide`	`DecideRequest`	`Decision`	The real product API. Writes to AuditLog unless `dryRun`
`POST /simulations`	`SimParams`	`SimResult`	Runs all four rule sets on the same seeded days (5 seeds, mean and range). Saves params + metrics, writes decisions to AuditLog
`GET /simulations/{id}`	none	`{ params, metrics per policy }`	No frames (too big to store)
`GET /audit`	`simulationId`, `riderId`, `policyVersion`, `outcome`, `limit` (default 50), `cursor`	`{ items: Decision[], nextCursor?: string }`	Newest first

Method + path	Body / query	Returns	Notes
`GET /policies`	none	`{ version, text, rules: { id, outcome, message } [] }[]`	Shows the Cedar source on screen

Errors always return `{ error: { code: string, message: string } }` with status 400 (bad input) or 500.

6. DynamoDB tables

Table	Keys	Other attributes	Access patterns
`HeatGateAudit`	PK `pk` = `SIM#<simulationId>` or `LIVE`; SK `sk` = `<at>#<decisionId>`	All `Decision` fields; `expiresAt` (TTL, 30 days: demo only; a regulator tier would need longer retention and a legal basis)	List a simulation's decisions newest first; filter by outcome or policy
`HeatGateAudit` GSI `byRider`	PK `riderId`; SK `sk`	(all)	One rider's full history
`HeatGateSimulations`	PK `simulationId`	`params`, `metrics` (per policy), `createdAt`	Load a past run

A simulated day creates a few thousand decisions per rule set. Write them with `BatchWriteItem` (25 per call) and give the Lambda a 30-second timeout and 1024 MB memory.

Core logic

Four small pieces of logic make the whole thing work. Each one is a pure function you can test without AWS.

Heat index (`engine/heatIndex.ts`)

Use the standard NOAA heat index formula (the Rothfusz regression, which works in °F, so convert in and out). Below about 27 °C, just return the air temperature.

Sanity checks for the test file: 35 °C at 50% humidity should come out at about 41 °C, and 32 °C at 40% humidity at about 34 °C. Allow ±1.5 °C.

Heat index is a stand-in for what an outdoor worker feels: it assumes shade and light wind. WBGT (ISO 7243) is the better measure for outdoor work because it includes sun and surface heat. Say so in the pitch; WBGT is a stretch goal.

Temperature through the day (`sim/city.ts`)

The slider sets the afternoon peak. Temperature follows a smooth curve that peaks at 3 PM and is about 9 °C cooler at 3 AM:

```
T(h) = T_{peak} - 9 \cdot \frac{1 - \cos\left(2\pi (h - 15)/24\right)}{2}
```

Humidity stays fixed at the slider value. Every minute, heat index = `heatIndex(T(h), humidity)`.

Alert level for the day: use the setting in `SimParams.alertLevel`. On `auto`, it is 1 when the peak is 40 °C or more and 2 at 45 °C or more. The same level applies all day, like a daily IMD alert.

The city and the riders

- A 10 × 10 km square. 6 restaurant hubs and 8 cooling points (shade + water), at fixed seeded positions. Cooling points are the demo's rest places. With 8 points in 100 km² the nearest one is about 1.8 km away on average, so the ride there counts as outdoor time and the rest starts on arrival.
- 50 riders: 70% bike (18 km/h), 20% EV (18 km/h), 10% cycle (12 km/h). Random start positions.
- **Shifts.** Each rider logs in at a random time between 8:00 and 13:00 and works `shiftHours` (6 to 8 hours), so the number of riders on duty rises and falls through the day.
- **Multi-app riders.** `multiAppPct` of riders (picked by the seed, default 40%) keep working on another app while they rest or are paused here. For them, `minutesSinceBreak` and exposure keep running during the rest and are not reset when it ends, and they cannot take our orders until it ends.

- Orders arrive randomly at `ordersPerHour` (default 50), 1.5× busier from 12-2 PM and 7-9 PM. Pickup at a hub, drop within 5 km.
- `tripMinutes` = travel to pickup + travel to drop + 5 minutes handover.
- Use a seeded random generator (for example mulberry32). Never `Math.random`, or the rule sets will not see the same day.
- **Tune on Day 1.** Print the demand level on the no-rules run. Aim for about 20% quiet, 50% normal and 30% rush. With staggered shifts, 50 riders on 25-minute trips deliver roughly 70 orders an hour, so a demand of 150 an hour would be overload all day and the "quiet time" rules would never fire.

The dispatch loop (`sim/dispatcher.ts`), one step per simulated minute

1. Log in riders whose shift starts now and log out riders whose shift has ended. Set the day's alert level.
2. Work out the demand level: the share of on-shift riders who are delivering. Below 50% → 0 (quiet), 50–85% → 1 (normal), above 85% → 2 (rush).
3. Count down rests. A rider whose rest hits 0 becomes idle, and `minutesSinceBreak` resets to 0 (not for multi-app riders, see above).
4. Move delivering riders along their route. On drop-off they become idle, get paid, and their delivery time is recorded.
5. For each waiting order, oldest first: take the 5 nearest idle riders and call `decide()` for each, in order.
   - `ALLOW` → assign the order, stop.
   - `SKIP_ORDER` → rider stays idle and available. Try the next rider.
   - `FORCE_REST` → rider rides to the nearest cooling point (this ride counts as outdoor time) and rests `restMinutes` from arrival. Try the next rider.
   - `DENY` → rider is paused for 15 minutes, then can be asked again. Try the next rider.
   - Nobody allowed → the order waits until the next minute.
6. Add 1 to `minutesSinceBreak` and `exposureMinutesToday` for every idle or delivering rider (they wait outdoors). Paused and resting riders count as on break, except multi-app riders.
7. Every 5 minutes, save a `Frame`.

Run this loop once per rule set and seed, using the same seeds for every rule set (default 5), and compute `Metrics` at the end of each run. Report the mean and the min–max range. If two rule sets' ranges overlap on a metric, say there is no clear difference instead

of ranking them.

Fair features and rider earnings (`sim/fair.ts`, `sim/earnings.ts`)

Earnings and pay protections apply to every rule set, so they can be compared fairly. Which protections each rule set has is set by `DEFAULT_PAY` in `contracts/types.ts`: NDMA has paid rest; the union and Fair have paid rest and incentive protection; today has neither. These defaults are assumptions to verify. Only two features are specific to `heatgate-fair`.

Earnings (all rule sets)

- Each delivered order pays `payPerOrderInr + payPerKmInr × distanceKm`.
- **Daily target.** The target is `dailyTargetShare` (default 0.9) times the median orders per rider in the no-rules run on the same seed, so it is reachable but not trivial. A rider who reaches it gets `dailyBonusInr`.
- **Incentive protection** (where on): every 30 minutes of forced rest or pause lowers that rider's target by 1 order.
- **Paid rest** (where on): rest minutes are paid at `restPayPerMinuteInr`.
- Show each rule set's average earnings as a % of the "today" baseline, and the lowest-earning rider. That answers "does safety cost riders money?"

Fair features (heatgate-fair only)

1. **Heat fee** (heatgate-fair only). Every order placed while the heat index is 38 or more adds `heatFeePerOrderInr` to a heat fund that pays the rest minutes. Each rupee of fee also cancels `feeDemandDropPctPerInr` % of new orders (default 1%), because customers may order less. The metrics show whether the fund covers the rest pay. This is a hypothesis, not a result, and it may stack with state welfare fees we have not checked.
2. **Shade waiting counts as rest** (heatgate-fair only, off by default). Idle riders wait at the nearest restaurant hub instead of the roadside, no exposure is added there, and 20 continuous minutes count as a break. This is only true if hubs really have shade for riders, which is unverified, so leave it off unless the rider interviews say otherwise. If you turn it on, say so wherever you show results.

Write each feature behind its own flag in `FairFeatures`, so you can switch either off. We dropped "first in line after rest": riders cannot see it, so it adds little trust.

Golden test cases (`engine/decide.test.ts`)

These feed the Cedar context directly, so they test the rules, not the heat formula. Write these tests before the engine; when they pass, the rules are right.

#	Heat index	Hour	Since break	Trip min	Dist km	Rest left	Demand	Alert	ndma-2025	union-demand	heatgate-fair
1	35	13	90	20	2	0	1	0	ALLOW	ALLOW	ALLOW
2	42	13	30	20	2	0	1	1	DENY (ndma-peak-hours-pause)	ALLOW	ALLOW
3	42	17	90	20	2	0	1	1	FORCE_REST 15 (ndma-break-due)	ALLOW	FORCE_REST 20 (fair-rest-due)
4	47	9	10	15	2	0	1	2	ALLOW	ALLOW	DENY (fair-extreme-heat-stop)
5	39	14	0	15	2	12	1	1	FORCE_REST 12 (ndma-rest-in-progress)	FORCE_REST 12 (union-rest-in-progress)	FORCE_REST 12 (fair-rest-in-progress)
6	47	13	90	20	2	0	1	2	FORCE_REST 15 (ndma-break-due; rest beats deny)	ALLOW	FORCE_REST 20 (fair-rest-due; rest beats deny)
7	42	13	90	20	2	0	2	1	FORCE_REST 15 (ndma-break-due)	ALLOW	ALLOW (rest delayed in rush)
8	42	13	110	20	2	0	2	1	FORCE_REST 15 (ndma-break-due)	FORCE_REST 20 (union-rest-due)	FORCE_REST 20 (fair-rest-hard-limit)
9	42	17	30	35	5	0	1	1	SKIP_ORDER (ndma-reduced-radius)	ALLOW	SKIP_ORDER (fair-short-trips-only)
10	39	16	75	10	1	0	0	0	ALLOW	ALLOW	FORCE_REST 20 (fair-rest-early-in-lull)
11	44	13	130	20	2	0	1	0	ALLOW (no alert)	ALLOW (no alert)	FORCE_REST 20 (fair-rest-due; fair-rest-hard-limit also fired)

`no-rules` returns ALLOW for every case.

Also check `penaltyWaived`. Case 1: false for all four rule sets (no alert, heat index below 38). Case 2: an ALLOW under union-demand or heatgate-fair gives true. `no-rules` is always false. NDMA and the union waive the penalty only on alert days; Fair waives it whenever the heat index is 38 or more. Also test the distance rounding: a 3.0 km order is

allowed by NDMA's radius rule on an alert day, and a 3.2 km order is skipped, because the distance is rounded up to 4 before it reaches Cedar. Case 11 shows the gap in today's advisory: on a very hot day with no alert, NDMA and the union do nothing.

Stand-out features (Day 3)

Three features turn a good demo into a memorable one: judges change the rules themselves, the numbers come from a real heatwave day, and the platform's business case is on screen. Build them only on Day 3, after the core and HeatGate Fair are deployed and working.

1. Change the rule live

A judge moves a slider, such as the rush-hour hard limit from 120 minutes down to NDMA's 90, clicks Run, and sees safety, speed and earnings change in seconds. The generated Cedar code is shown next to it, with the changed numbers highlighted.

To stay safe, judges edit **numbers, not free text**. The numbers are filled into a template, then Cedar checks the result against the schema before anything runs, so a bad value can't crash the demo.

```
// contracts/types.ts (add)
export interface FairKnobs {
  restHeatC: number;         // default 38, allowed 32-46
  workLimitMin: number;      // default 100, allowed 60-120
  rushHardLimitMin: number;  // default 120, allowed 90-150, must be >=
workLimitMin
  earlyRestMin: number;      // default 70, allowed 40-110, must be <
workLimitMin
  shortTripHeatC: number;    // default 41, allowed 36-48
  shortTripMaxKm: number;    // default 3 (NDMA's radius), allowed 1-6
  extremeHeatC: number;      // default 46, allowed 42-52
}
// SimParams gets: fairKnobs?: FairKnobs   (missing = defaults)
```

- `policies/heatgate-fair.cedar.tmpl` is the Fair rule set with `{{restHeatC}}`-style placeholders. The engine fills it in, validates it, and caches it by knob values.
- New route `POST /policies/render` with body `{ knobs: FairKnobs }` returns `{ text: string, valid: boolean, errors: string[] }`. Out-of-range values return 400.

- `POST /simulations` accepts `fairKnobs`. To keep it fast, a knob change reruns only `heatgate-fair` and `no-rules`.
- Test: rendering the default knobs must give exactly the committed `heatgate-fair.cedar`, so the golden tests still cover it.
- On screen: show only 3 sliders (rest heat, work limit, rush hard limit). Keep the rest at defaults. Mark NDMA's 90 and the union's 120 on the work-limit and hard-limit sliders.

2. Replay a real heatwave day

Instead of a made-up slider day, replay the real hourly temperature and humidity of a real hot day in a real city. Then the pitch says "on this real day, here's what each rule would have done."

```
// contracts/types.ts (add)
export interface WeatherDay {
  id: string;              // e.g. "delhi-2024-xx-xx"
  city: string;
  date: string;            // YYYY-MM-DD
  lat: number; lng: number;
  source: string;          // "Open-Meteo historical weather API"
  alertLevel?: 0 | 1 | 2;  // the real IMD alert if you can find it;
otherwise derived from the day's peak and shown as "assumed"
  hourly: { hour: number; tempC: number; humidityPct: number }[]; // 24
entries
}
export type WeatherSource =
  | { kind: "slider" }                        // uses peakTempC + humidityPct
  | { kind: "replay"; weatherDayId: string };
// SimParams gets: weather: WeatherSource
```

- `scripts/fetch-weather.ts` calls Open-Meteo's free historical weather API once for hourly temperature and relative humidity (timezone Asia/Kolkata) and saves `sim/weather/<id>.json`. **Commit these files**, so the demo never needs the internet.
- Pick the day from the data, not from memory: fetch a whole summer (for example May–June) for 2–3 cities and choose the day with the highest peak heat index. Check the date against news reports before quoting it.
- Between hours, the simulator blends the two nearest hourly values.
- New route `GET /weather-days` returns `{ id, city, date }[]` for a dropdown.
- Use your own city too (Hyderabad), since it connects with local judges.

3. Platform view

Judges will ask "why would Swiggy or Zomato agree to this?" One small screen answers it, computed from the metrics you already have. No new backend.

Number on screen	How it's computed (heatgate-fair vs the today baseline)
Rest pay vs heat fee	`restPayTotalInr` next to `heatFeeCollectedInr`, and whether the fee covers the pay
Safety cost per order	(rest pay − heat fee) ÷ orders delivered, in ₹
Delivery time change	% change in `avgDeliveryMinutes`
Late orders change	change in `ordersLate`
Unsafe heat prevented	drop in `unsafeExposureMin120` (and `unsafeExposureMin90`), in rider-hours

All rupee amounts are still placeholders until your rider interviews replace them. Label this on screen. The heat fee is a hypothesis: customers may order less (the simulator has a demand-drop setting), and it may stack with state welfare fees that we have not verified.

Only if Day 3 morning finishes early

- **Judge becomes a rider:** a QR code opens a phone page for Rider #51 that checks the API every 3 seconds and shows the rest card live. *(Half a day)*
- **Tamper-proof audit log:** each record stores a hash of the record before it; a "Verify" button turns red if any old record was changed. *(2–3 hours)*

Do not start either one after 12:00 on Day 3.

Later, or pitch only

- **Heat-emergency button:** the rider pauses themselves with no penalty. NDMA's advisory asks for a way to do this.
- **WBGT** as an optional trigger instead of the heat index.
- **Quick-commerce profile:** 3 km radius and 10-minute trips with its own rest settings, because a 20-minute rest breaks a 10-minute delivery promise.
- **Cross-platform limit:** one shared heat clock across apps, run by a regulator-level service.

- **Tiered log access:** aggregate reports for unions, exact records only for regulators with a legal basis, and the ID key held by a trustee.

3-day schedule

Each day ends with something that works and is tagged in Git. Day 1: the core works on a laptop. Day 2: HeatGate Fair works and is online. Day 3: stand-out features until 1 PM, then freeze, rehearse and submit.

The plan assumes three people: **A** owns the engine, API and AWS; **B** owns the simulator; **C** owns the web app. With two people, merge B into A. Solo, skip the Day 3 platform view and both "only if early" features.

Day 1: the core works locally

09:00–10:00 · Everyone together: contracts

- [ ] Create the repo, npm workspaces, MIT license
- [ ] Write `CLAUDE.md` (see Building with Claude)
- [ ] Commit `contracts/types.ts`, the Cedar schema and all four `.cedar` files exactly as in this doc. If an older copy is already committed, replace it first (see the repo update prompt in Building with Claude)
- [ ] Check the policies parse and validate against the schema with `@cedar-policy/cedar-wasm`

10:00–13:00 · Build the three parts in parallel

- [ ] A, first hour: deploy an empty skeleton to AWS (Lambda returning "ok" behind API Gateway, a blank page on S3). Every later deploy is then a small change
- [ ] A: `heatIndex.ts` + `decide.ts` + all 11 golden tests passing
- [ ] B: city, riders with staggered shifts, seeded orders, demand level (share of riders busy), dispatch loop using a fake `decide()` that always says ALLOW
- [ ] C: React app with 3 screens, fed by a hand-written `fixtures/simResult.json` that matches `SimResult`

14:00–17:00 · Connect the pieces

- [ ] A: `api/local.ts` Express server with `POST /decide` and `GET /policies`, in-memory store

- [ ] A: `npm run smoke`, a script that calls every route and checks the answers. Run it before every commit from now on
- [ ] B: swap in the real `decide()`, add the safety and speed metrics, expose `POST /simulations`
- [ ] C: sandbox screen: slider, policy selector, SVG map that replays frames

17:00–20:00 · Finish the screens

- [ ] A: `GET /audit` with filters and paging (memory store), pseudonymous rider IDs only
- [ ] B: sanity check: at peak 44 °C with the alert on, union-demand should show fewer unsafe minutes than no-rules, and ndma-2025 should show a delivery stall between 11 and 4. With alert level 0, ndma-2025 and union-demand should match no-rules exactly
- [ ] B: tune `ordersPerHour` and shift lengths until the no-rules run is quiet about 20% of the day, normal about 50% and rush about 30% (print the histogram), and the system does not collapse at 34 °C
- [ ] C: rider phone view and the audit log table

20:00–21:00 · Checkpoint

- [ ] Web app talks to the local API end to end; `npm run smoke` passes
- [ ] Click a red dot on the map → rider view → its audit entry shows the rule that fired
- [ ] Commit and tag `day1-working`

Day 2: HeatGate Fair, and online

09:00–13:00 · Fair and the real backend

- [ ] A: full `template.yaml`, DynamoDB store behind the same interface as the memory store, deploy
- [ ] B: rider earnings and pay protections for all rule sets (DEFAULT_PAY), multi-apping (multiAppPct), 5-seed runs with ranges, then Fair's heat fee with its demand drop. Shade waiting stays off unless the rider interviews confirm it
- [ ] C: comparison table for all four rule sets (safety, speed, money) with the heatgate-fair column highlighted; Cedar source panel

14:00–17:00 · Polish on the live URL

- [ ] A: frontend built and on S3, pointing at the deployed API; `npm run smoke` against the live URL

- [ ] B: check whether heatgate-fair keeps exposure low with earnings close to the today baseline; tune if not. Then the heat sweep data (peak 34 to 46 °C, all four rule sets)
- [ ] C: rider view in Hindi, Telugu and English (Urdu if a native speaker helps) with large text: next order, skipped-order notice, paid rest card with cooling point. Heat sweep chart: rider earnings vs unsafe exposure

17:00–19:00 · Checkpoint

- [ ] Full flow works on the live URL, from a phone and a second laptop
- [ ] Commit and tag `deployed-v1`

Evening · Rider interviews (1 hour)

- [ ] Talk to 3–5 delivery riders nearby (questions in Built for Indian riders)
- [ ] Replace the placeholder money amounts in `economics` with what you heard. Also set `multiAppPct`, `dailyTargetShare` and the shade-waiting flag from their answers. Note 2–3 quotes for the pitch (no names)
- [ ] Ask native speakers to write and check the Hindi and Telugu messages (and Urdu if possible)

Day 3: stand out, then finish

09:00–13:00 · Stand-out features

- [ ] A: change the rule live: template, `POST /policies/render`, knob validation, default-knobs test
- [ ] B: real heatwave replay: fetch script, 2–3 saved weather days (including Hyderabad), replay in the simulator, `GET /weather-days`
- [ ] C: knob sliders + highlighted Cedar panel, weather-day dropdown, platform view
- [ ] Only if all of the above is done by 12:00: one of the "only if early" features

13:00–14:00 · Deploy and freeze

- [ ] Deploy, `npm run smoke` on the live URL, tag `final`
- [ ] Feature freeze at 14:00. Only fixes after this

14:00–16:00 · Rehearse

- [ ] README: pitch, screenshots, how to run locally, the four rule sets, thresholds and money disclaimer
- [ ] Three timed run-throughs of the demo script; one of them with Wi-Fi off, using the saved offline result

16:00–17:30 · Record

- [ ] Record the 3-minute demo video, two takes, and pick the better one

17:30–19:00 · Submit

- [ ] Repo public, demo URL working, video uploaded
- [ ] Submission form filled, with at least an hour to spare before the deadline

Building with Claude

Use Claude Code in the repo. Claude reads `CLAUDE.md` at the start of every session, so the contracts live there once and every prompt stays short. Give Claude one block of work at a time, ask it to run the tests, and commit after each block that works.

`CLAUDE.md` (put this in the repo root)

```
# HeatGate

Heat-safety rules for delivery riders, written as Cedar policies and run
inside order dispatch.
Core idea: protect riders WITHOUT making safety cost them money.
Hackathon project: 3 days. Prefer simple, working code over clever code.

## Contracts (do not change without asking)
- Shared types: contracts/types.ts. Import from here everywhere.
- Cedar schema: policies/heatgate.cedarschema (context includes demandLevel
and alertLevel)
- Rule sets: policies/no-rules.cedar (today: voluntary measures, nothing
enforced),
  ndma-2025.cedar, union-demand.cedar, heatgate-fair.cedar
- ndma-2025 and union-demand copy the real NDMA advisory and the IFAT demand
and trigger on
  alertLevel. Do not tweak their numbers. If one looks wrong, tell me.
- Every forbid rule has @id, @outcome (SKIP_ORDER | DENY | FORCE_REST) and
@message.
  FORCE_REST rules also have @restMinutes (20 if missing). Optional:
@messageHi, @messageTe,
  @messageUr.
- Decision logic: AssignOrder allowed -> ALLOW. Denied -> strongest outcome
among fired rules:
  FORCE_REST > DENY > SKIP_ORDER. Ties: the first rule in policy file order
```

```
wins (message and
  rest length); firedRules lists the winner first. Second check
DeclineWithoutPenalty ->
  penaltyWaived (always true for SKIP_ORDER, DENY and FORCE_REST).
- Pay protections per rule set live in DEFAULT_PAY (contracts/types.ts).
Fair's two features
  (heat fee, shade waiting) live in sim/fair.ts behind FairFeatures flags.
Shade waiting is
  OFF by default.
- All money amounts are placeholders from SimParams.economics. Never hard-
code rupees.
- Heat index is a proxy for outdoor heat stress. It is not a medical measure.

## Rules
- TypeScript strict mode, Node 20, npm workspaces.
- Cedar runs in-process with @cedar-policy/cedar-wasm. Read its README for
the exact API
  of the installed version before writing calls.
- Heat index is rounded to a whole number and order distance is rounded UP to
a whole km
  before they go to Cedar (Cedar uses Long).
- Randomness only through the seeded RNG in sim/rng.ts. Never Math.random.
- Every simulation runs over several seeds (default 5). Report the mean and
the range.
- Rider IDs in the audit log are pseudonymous (likely still personal data
under DPDP).
  Never store names or phone numbers.
- The store has one interface (api/store/types.ts) with two versions: memory
and DynamoDB.
- Tests: vitest. Run `npm test` after every change and fix failures before
finishing.
- No new dependencies without saying why.

## Layout
engine/ heat index + decide()   sim/ simulator + fair features + earnings
api/ handlers + stores          web/ React app
```

Prompts, in order

Copy each one into Claude Code at the matching block of the schedule.

Prompt 0: if you already committed the first version of the contract files. Run this before anything else, and paste the new file contents from this doc under it.

> The plan changed after we committed our contract files. Replace these files with the versions I'm pasting, exactly as given: contracts/types.ts, policies/heatgate.cedarschema, policies/no-rules.cedar, policies/ndma-2025.cedar, policies/union-demand.cedar, policies/heatgate-fair.cedar, and CLAUDE.md. Delete policies/heatgate-fair.cedar.tmpl for now; it is a Day 3 item and will be regenerated from the new Fair file. Then run npm run check-policies. If the script breaks because of the new alertLevel context field or the @restMinutes annotation, fix the script (it must accept @restMinutes on FORCE_REST rules and optional @messageHi, @messageTe and @messageUr). Show me the output. Do not change any policy or type yourself; if something fails validation, tell me which line.

1. **Contracts (Day 1, 09:00)**

   > Set up an npm workspaces monorepo with folders contracts, engine, sim, api, web, using TypeScript strict and vitest. I'm pasting our contract files: types.ts, the Cedar schema and four policy files. Save them exactly as given. Then install @cedar-policy/cedar-wasm, read its README, and write a script `npm run check-policies` that parses both policy files and validates them against the schema. Run it and show me the result.

2. **Engine (A, 10:00)**

   > Build engine/heatIndex.ts using the NOAA Rothfusz heat index formula, input and output in °C. Then build engine/decide.ts: `decide(req: DecideRequest): Decision`, following the decision logic in CLAUDE.md. Load all four policy sets once at startup and read the @id, @outcome, @restMinutes and @message annotations (and @messageHi, @messageTe, @messageUr if present) from them. Keep each file's rule ids in file order; when several rules of the strongest outcome fire, the first in file order supplies the message and rest length. Round the heat index to the nearest whole number and the order distance up to a whole km before calling Cedar. Write engine/decide.test.ts with these 11 golden cases first [paste the golden test table], plus the heat index sanity checks. Make all tests pass.

3. **Simulator (B, 10:00)**

> Build the simulator in sim/ following this spec [paste the Core logic section: city, riders, dispatch loop]. Use a seeded mulberry32 RNG in sim/rng.ts. For now, take `decide` as a function parameter and test with a fake that always returns ALLOW. Export `runSimulation(params: SimParams, decide): SimResult`. Add a test that the same seed gives identical results twice. Then add staggered shifts, multi-apping, rider earnings, pay protections (DEFAULT_PAY) and 5-seed runs with mean and range for every rule set, and Fair's two features (heat fee, and shade waiting which stays off by default), exactly as in this spec [paste the city and riders part, the dispatch loop and the Fair features and rider earnings part].

4. **Web app (C, 10:00)**

> Build a React + Vite + Tailwind app in web/ with three screens: Policy Sandbox, Rider View, Audit Log. Use the types from contracts/types.ts. Create web/fixtures/simResult.json with a small realistic fake SimResult (10 riders, 20 frames, all four policies) and drive all screens from it for now. Sandbox: peak temperature slider (34–46 °C), policy toggle, an SVG map of a 10×10 km city that replays frames (dot colour by rider state: idle grey, delivering blue, resting amber, paused red), and a comparison table of all four policies with three groups of columns: safety (unsafe exposure minutes beyond 90 and beyond 120, with the min–max range), speed (average delivery time) and money (average and lowest rider earnings, incentive hit rate). Highlight the heatgate-fair column. Keep it clean and readable on a projector.

5. **Local API (A, 14:00)**

> Build api/ handlers for the 5 routes in this table [paste the REST API table], each as a plain function `(event) => response` so it works in Lambda later. Add api/local.ts, an Express server that wraps those handlers on port 3001, and api/store/memory.ts implementing the store interface. Validate request bodies and return `{ error: { code, message } }` with 400 on bad input.

6. **Wire up (B and C, 14:00)**

> Connect sim to the real engine decide(). Add the Metrics calculation exactly as defined in contracts/types.ts. Make POST /simulations run all four policies with the same 5 seeds and write all decisions to the store. Then point the web app at http://localhost:3001 instead of the fixture, with a loading state while the simulation runs.

7. **Deploy (A, Day 2, 09:00)**

> Write a SAM template.yaml: one Node 20 Lambda (1024 MB, 30 s timeout) behind an HTTP API with CORS for our frontend, two DynamoDB on-demand tables as defined in this doc [paste the DynamoDB table], with the byRider GSI and TTL on expiresAt. Implement api/store/dynamo.ts with the same interface as the memory store, using BatchWriteItem in parallel batches of 25. Give me the exact commands to build and deploy.

8. **Before recording (anyone, Day 3, 14:00)**

> Review the whole repo as a hackathon judge would. List the top 5 things that could break during a live demo and fix them. Then write the README: one-paragraph pitch, screenshot placeholder, how to run locally, how the two rule sets differ, and a clear note that heat thresholds are placeholders pending expert review.

Day 3 prompts (stand-out features, 09:00)

- **Change the rule live (A)**

> Add FairKnobs to contracts/types.ts exactly as in this spec [paste the "Change the rule live" part]. Turn policies/heatgate-fair.cedar into a template policies/heatgate-fair.cedar.tmpl with a placeholder for each knob. In the engine, render the template from knobs, validate it against the schema with cedar-wasm, and cache it by knob values. Add POST /policies/render and accept fairKnobs in POST /simulations (rerun only heatgate-fair and no-rules). Reject out-of-range knobs with 400. Add a test that the default knobs render exactly the committed heatgate-fair.cedar, and keep all golden tests passing.

- **Real heatwave replay (B)**

> Write scripts/fetch-weather.ts that calls the Open-Meteo historical weather API for hourly temperature_2m and relative_humidity_2m, timezone Asia/Kolkata, for a date range and a lat/lng. For each city I give you, fetch May 1 to June 30 of a recent year, compute the daily peak heat index with engine/heatIndex.ts, print the 5 hottest days, and save the hottest as sim/weather/<city>-<date>.json in the WeatherDay format [paste type]. Then add WeatherSource to SimParams and make the simulator use replayed hourly values, blending between hours. Add GET /weather-days. The demo must work offline from the saved files.

- **Platform view and knob UI (C)**

> Add three things to the web app. (1) A Rules panel with 3 sliders (rest heat, work limit, rush hard limit) that calls POST /policies/render, shows the generated Cedar with changed numbers highlighted, and has a Run button. (2) A weather dropdown from GET /weather-days, plus "Custom (slider)". (3) A Platform View screen that computes, from the metrics we already return, heatgate-fair vs the today baseline: rest pay vs heat fee, safety cost per order in ₹, % change in delivery time, change in late orders, and unsafe heat prevented in rider-hours. Label all rupee values as placeholders.

Tips that save hours

- When Claude gets something wrong twice, stop and paste the exact error plus the relevant contract. Do not let it guess.
- Ask for one file or one feature at a time. Big "build everything" prompts produce code nobody understands at 2 AM.
- If the cedar-wasm API is confusing, ask Claude to write a 10-line script that calls `isAuthorized` once with a hard-coded request and prints the full response. Build on what works.

Demo script, risks and submission

3-minute demo script

Time	On screen	What you say
0:00–0:25	Title slide or a photo of a rider in afternoon heat	"Delivery riders work through heatwaves. NDMA's 2025 advice on breaks was only a recommendation, and unions are asking for a 20-minute cooling break every 2 hours. The rules exist on paper. Nothing enforces them where the work is assigned: inside the dispatch app. And in India, a break that costs a rider money simply won't be taken."
0:25–0:50	Cedar source panel	"HeatGate writes those rules as code, using Cedar, an open-source policy language normally used for app permissions. Here are the NDMA advisory rules, the union demand, and our own rule set, HeatGate Fair. NDMA's advisory is only a recommendation today; this is what enforcing it would look like."
0:50–1:40	Sandbox: pick the real heatwave day from the dropdown, run all four rule sets, watch heatgate-fair	"50 riders on a real heatwave day: [city, date]. Watch the amber dots: those riders were stopped before taking an order, sent to the nearest shade and water, and their rating is protected." Click one amber dot.
1:40–2:05	Rider phone view, then its audit entry	"This is all the rider sees. And this is what a union or regulator sees: the exact rule that fired and the exact inputs, so any decision can be checked later."

Time	On screen	What you say
2:05–2:40	Side-by-side metrics, then the heat sweep chart	"Same day, four rule sets. Read what the numbers actually show: which rule sets cut unsafe hours, what each costs in delivery time, and what each does to rider earnings, with the ranges. HeatGate Fair's idea is to move rest into quiet time and pay for it; say plainly if the numbers don't support that. Here's unsafe exposure, delivery time and rider earnings for each. Safety that doesn't cost the rider a rupee of their day." Read your real numbers, and only use the closing line if they support it. Then move the rush-limit slider and rerun, so judges see the rule change live.
2:40–3:00	Architecture slide	"It's one API call any delivery platform can add to dispatch, running on AWS Lambda. Unions, researchers and policy designers can test a rule here before it becomes law. It is the enforcement and audit layer on top of what platforms already do. Thresholds are configurable and need expert review."

Rehearse it with a timer. If you run long, cut the architecture slide, never the audit log moment.

Risks and fallbacks

Risk	Fallback
cedar-wasm API differs from what Claude expects	Hour 1: get a 10-line script working first. If stuck, run the Cedar CLI (Rust) in a child process locally and demo locally

Risk	Fallback
Simulation is slow on Lambda	Lower to 30 riders or 10-minute frames; or run the simulator in the browser (it is pure TypeScript) and only call `/decide` remotely
Results look wrong or boring	Tune `ordersPerHour` and the temperature curve on Day 1 evening; check the golden tests still pass
AWS deploy fails	Keep the `day1-working` tag. Demo from localhost and record the video from it
Wi-Fi fails during the live demo	Pre-recorded video as backup; a saved `simResult.json` the web app can load offline
A judge questions the thresholds	Say plainly they are placeholders, configurable per rule set, and must be set by health experts
A judge asks "is Fair's 120-minute hard limit safe?"	Say it is a policy number: equal to the union's demand and 30 minutes looser than NDMA's 90. Health experts decide. The knob slider shows the trade-off
A judge knows the NDMA advisory and spots a wrong number	The rules follow the advisory (alert-triggered, 15 per 90, 3 km). Say plainly where numbers came from news coverage, and fix any that differ once you have read the primary text
"Platforms already do this"	Agree that some add rest points and coolers. HeatGate is the enforcement, audit and cost layer on top of voluntary measures. Only say this after you have checked each platform's current measures
"A rider can just open another app"	Show the multi-app setting and its effect. A cross-platform limit run by a regulator-level service is the next step
Differences between rule sets look like noise	Show the min–max range across 5 seeds. When ranges overlap, say "no clear difference" instead of ranking

Risk	Fallback
A question about rider data and privacy	Pseudonymous IDs are likely still personal data. Tiered access and a legal review under DPDP are next steps. The demo uses only simulated riders

Submission checklist

- [ ] Public GitHub repo with MIT license and README
- [ ] Live demo URL works from a phone
- [ ] 3-minute demo video uploaded (unlisted YouTube is fine)
- [ ] README explains how to run locally in under 5 commands
- [ ] Screenshots: sandbox, rider view, audit log, Cedar policies
- [ ] Clear note that thresholds are placeholders
- [ ] Check the hackathon rules for team size, video length and any required tracks, and adjust this plan if they differ

Cedar is open source, which may count toward an open-source bonus. Check the hackathon rules for what qualifies before you claim it.